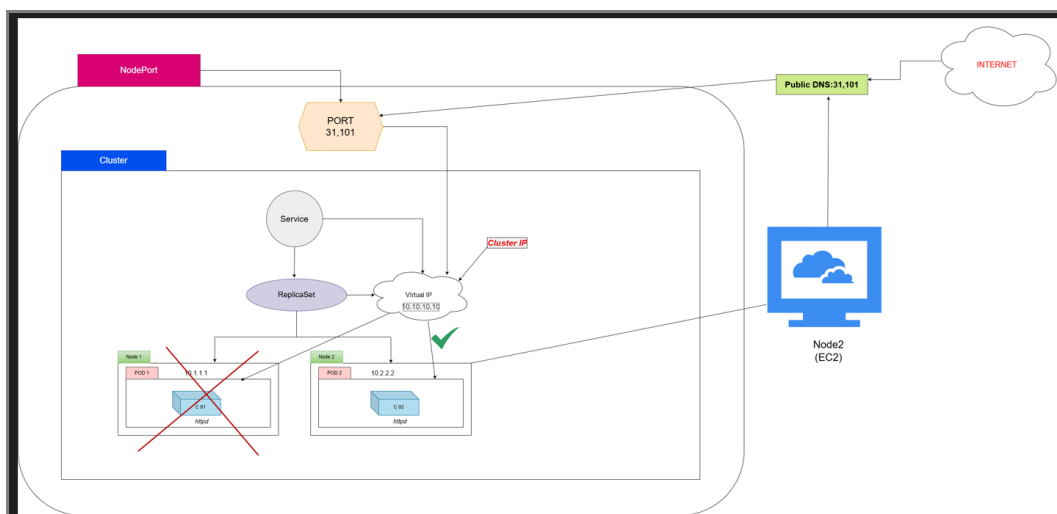

NodPort practical

Rohit Surwade

NodePort



1.

NodePort is a type of Kubernetes Service that exposes a pod or set of pods to external traffic by assigning a static port on each node in the cluster.

This allows external clients to access the service using the node's IP address and a specific port (the NodePort).

A manifest to create a Deployment object

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mydeployments
spec:
  replicas: 1
  selector:
    matchLabels:
      name: deployment
  template:
    metadata:
      name: testpod1
      labels:
        name: deployment
    spec:
      containers:
        - name: c00
          image: nginx
          ports:
            - containerPort: 80
```

2.

This manifest describes a Kubernetes Deployment named mydeployments that ensures one replica of a pod is running. The pod runs a container named c00 using the nginx image and exposes port 80. The deployment manages pods with the label name: deployment and ensures that the desired state (one running pod) is always maintained.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mydeployments
spec:
  replicas: 1 # Number of replicas (pods) to maintain
  selector: # Specifies the labels that identify the pods managed by this
  deployment
  matchLabels:
    name: deployment
  template: # Defines the pod template that is used to create the pods
  metadata:
    labels:
      name: deployment # This label should match the selector's matchLabels
  spec:
    containers:
      - name: c00 # Container name
        image: nginx # Image for the container
        ports:
          - containerPort: 80 # Port exposed by the container
```

Applying the manifest to create the Deployment object

```
root@ip-172-31-47-35:/home/ubuntu# vi deploynginx.yml
root@ip-172-31-47-35:/home/ubuntu# kubectl apply -f deploynginx.yml
deployment.apps/mydeployments created
root@ip-172-31-47-35:/home/ubuntu# kubectl get pods
NAME                                READY   STATUS              RESTARTS   AGE
mydeployments-648d7b5c49-7lhm7     0/1     ContainerCreating   0           8s
root@ip-172-31-47-35:/home/ubuntu# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
mydeployments-648d7b5c49-7lhm7     1/1     Running   0           12s
```

3.

Creating a Service object (NodePort)

```

root@ip-172-31-47-35:/home/ubuntu
apiVersion: v1
kind: Service                                # Defines to create Service type Object
metadata:
  name: demoservice
spec:
  ports:
    - port: 80                                # Containers port exposed
      targetPort: 80                          # Pods port
  selector:
    name: deployment                          # Apply this service to any pods which have the specific label
  type: NodePort                             # Specifies the service type i.e ClusterIP or NodePort

```

4.

- demoservice is a Service of type NodePort, which exposes the pods it targets (those with the label name: deployment) on port 80.
- The service listens on port 80, and any traffic to this port will be forwarded to the targetPort of the pods (also port 80).
- The NodePort type means that the service is accessible externally on any node's IP, with an automatically assigned port from the 30000-32767 range.
- This service allows external access to the pods running behind it by routing traffic through the NodePort, making the service accessible from outside the Kubernetes cluster, such as through a browser or curl command on the assigned port.

```

apiVersion: v1
kind: Service
metadata:
  name: demoservice
spec:
  ports:
    - port: 80 # Port exposed by the service
      targetPort: 80 # Port on the pods that the service forwards traffic to
  selector:
    name: deployment # Pods with this label will be targeted by the service
  type: NodePort # Exposes the service on a port on each node

```

Applying the manifest to create service object(i.e NodePort)

```

root@ip-172-31-47-35:/home/ubuntu# vi service.yml
root@ip-172-31-47-35:/home/ubuntu# kubectl apply -f service.yml
service/demoservice created
root@ip-172-31-47-35:/home/ubuntu# kubectl get svc

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
demoservice	NodePort	10.96.189.35	<none>	80:32510/TCP	7s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	30m

5.

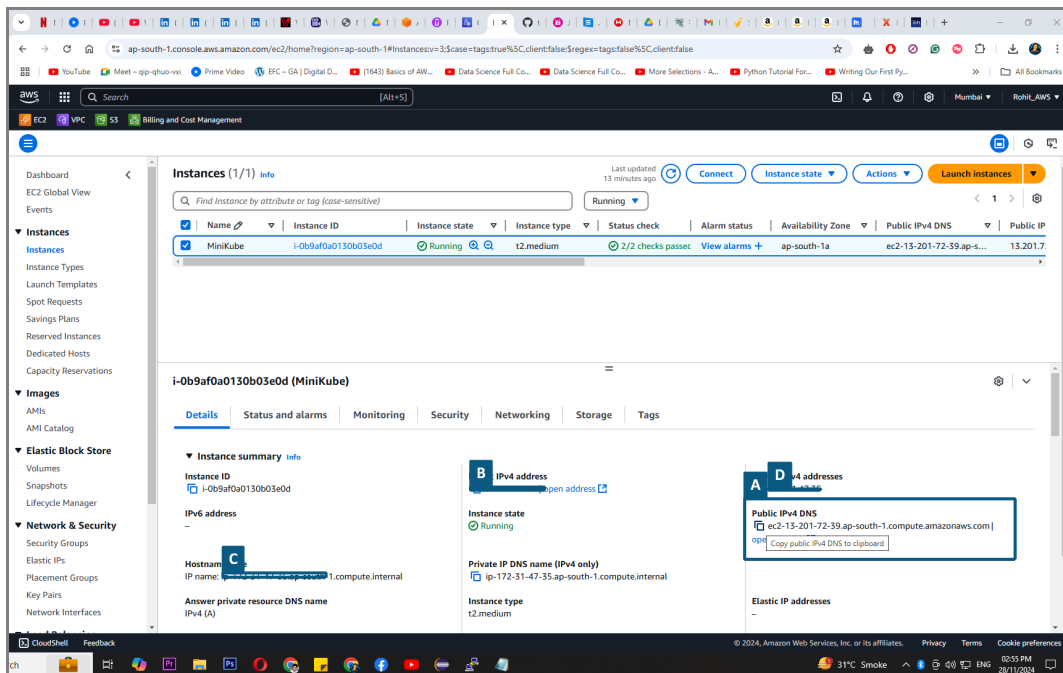
```
root@ip-172-31-47-35:/home/ubuntu# kubectl get svc
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
demoservice         NodePort    10.96.189.35    <none>            80:32510/TCP     4m32s
kubernetes           ClusterIP   10.96.0.1       <none>            443/TCP          35m
root@ip-172-31-47-35:/home/ubuntu# kubectl describe svc demoservice
Name:
Namespace:          default
Labels:             <none>
Annotations:        <none>
Selector:            name=deployment
Type:               NodePort
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                 10.96.189.35
IPs:                10.96.189.35
Port:               <unset> /TCP
TargetPort:         80/TCP
NodePort:           <unset> 32510/TCP
Endpoints:          172.17.0.3:80
Session Affinity:    None
External Traffic Policy: Cluster
Events:             <none>
root@ip-172-31-47-35:/home/ubuntu#
```

6.
kubectl describe svc service-name

NOTE:

- We have created a Deployment object which has one container(c 00) with nginx image running on it.
- NodePort has given us a port(An automatically assigned port from the 30000-32767 range) that is “32510”.

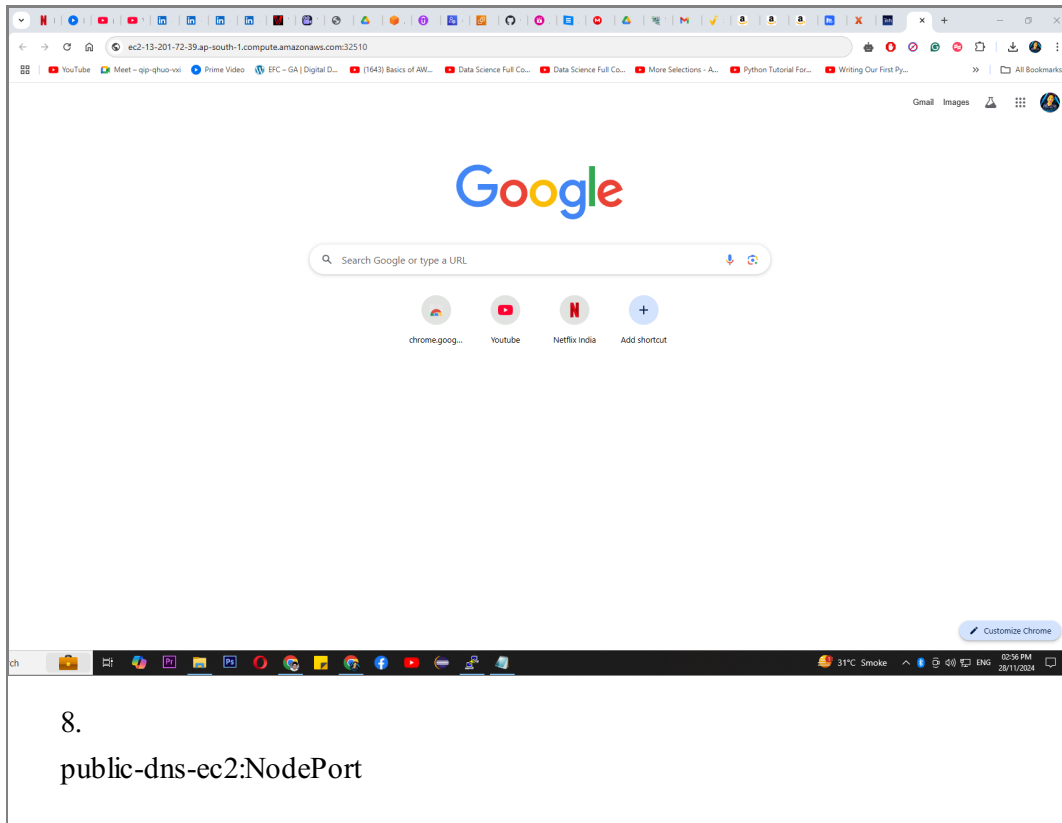
Copy the Public DNS of the EC2 instance



The screenshot shows the AWS Management Console for an EC2 instance named 'MiniKube'. The instance is in the 'Running' state. The 'Public IPv4 DNS' is highlighted with a blue box and labeled 'A'. The 'Public IPv4 address' is highlighted with a blue box and labeled 'B'. The 'Private IPv4 address' is highlighted with a blue box and labeled 'C'. The 'Private IPv4 DNS name' is highlighted with a blue box and labeled 'D'.

7.
WE SHOULD BE ABLE TO ACCESS CONTAINER USING public-dns-
ec2:NodePort

Paste public-dns-ec2:NodePort in new tab in the browser



VOILA! WE CAN ACCESS THE CONTAINER USING NODEPORT

